SIMPLE PROGRAMME · UNFPA BANGLADESH · HANDOVER PACK

Operations Runbook

Everything needed to keep SIMPLE alive: hosting, credentials, deployment, backups, recurring duties · Version 1.0 · August 2026 · Prepared by the M&E Advisor for CIPRB

1. What the system is

SIMPLE (simpledashboard.pro) is a Django 5 + React application on Railway with a PostgreSQL database. Field data arrives from 19 KoboToolbox forms via webhooks; managers approve records on the dashboard; approved records feed 40+ indicators, maps and an automated monthly reporting kit. Source code: `github.com/rafijahiin/spondon` (private).

2. Accounts that must survive the consultant

Account	What it controls	Action before November 2026
Railway (Hobby plan)	Hosting, database, domain, the $5/month bill	Transfer the project to a CIPRB-owned Railway account, or move billing to a CIPRB card. If the bill stops, the site stops.
KoboToolbox (kf.kobotoolbox.org)	All 19 forms, all raw submissions, the webhook REST services, the API token	Change the account email to an institutional CIPRB address; store the password in the office password safe.
GitHub repo `rafijahiin/spondon`	All source code; pushing to `master` deploys to production	Transfer the repository to a CIPRB organisation account, or add a CIPRB-controlled account as admin collaborator.
Domain simpledashboard.pro	The public address of the dashboard	Registered through Railway; renews yearly. Confirm renewal billing follows the Railway account transfer.
Groq (console.groq.com)	Free-tier API key for the AI narrative in monthly reports	Optional: create a CIPRB account and swap the key. If the key ever dies, reports still generate, only without the narrative paragraph.

3. Credentials and environment variables

All secrets live as environment variables on the Railway **web** service (never in code). View or edit: Railway dashboard → Simple dashboard → web → Variables, or CLI: `railway variables --service web`.

Variable	Purpose	Rotation rule
`FERNET_KEY`	Encrypts survivor and patient identifiers (GBV, fistula) at rest	**NEVER rotate, never lose.** See the red box below.
`SECRET_KEY`	Django session/CSRF signing	Rotating logs everyone out; otherwise safe.
`DATABASE_URL`	Postgres connection (internal)	Managed by Railway; do not hand-edit.
`KOBO_TOKEN`	Kobo API access: lookup-CSV uploads, form redeploys, ID write-backs	If rotated in Kobo (Account Settings → Security), update here within the hour, otherwise the client lookup lists stop refreshing.
`KOBO_WEBHOOK_SECRET`	Authenticates incoming Kobo webhooks	If rotated, update the Authorization header in EVERY form REST service in Kobo (project Settings → REST Services) at the same time.
`GROQ_API_KEY`	AI narrative in monthly reports	Safe to rotate any time.
`KOBO_ASSET_UID_*`	Optional overrides mapping Kobo assets to handlers	Leave as they are.

`USER_*_PASSWORD`	Seed passwords for the 13 dashboard accounts, used once at first boot	Changing them later has no effect; reset passwords from the dashboard instead.
`FLUSH_PRACTICE_DATA`	One-time practice-data wipe gate	**Leave unset.** Setting it wipes data at next boot.

> **FERNET_KEY is the single most dangerous variable in the system.** It decrypts the names, phone numbers and addresses of GBV survivors and fistula patients. If it is lost, that data is unrecoverable forever; if it is rotated, all previously stored identifiers become permanent gibberish. Keep one copy live in the Railway variables and one sealed offline copy with CIPRB management. It must never appear in code, email threads or chat messages.

4. How deployment works

```
git push origin master        # that is the whole deploy
# Railway auto-builds the Dockerfile (railway.json pins the DOCKERFILE builder),
# runs migrations + seeds on boot, then serves with gunicorn (1 worker, 4 threads).
```

- Verify a deploy by content: load simpledashboard.pro and check that the `assets/index-*.js` hash in the page source changed.
- Logs: `railway logs --service web`. Restart: Railway dashboard → web → Deployments → Redeploy.
- Rollback: redeploy the previous build from the Deployments list, or `git revert` and push.

5. Money

- Railway Hobby: $5/month flat fee that includes a $5 usage credit. Post-optimisation usage runs about $4-5/month, so the bill stays $5. Billing cycle: the 13th of each month.
- The compute usage limit is deliberately set to **$10**. Hitting the limit STOPS the services, so the limit is a fuse, not a budget; do not lower it.
- Check railway.app/account/usage weekly. Memory is the dominant cost; brief 1.5 GB spikes during monthly report generation are normal.

6. Backups

```
# Full database dump (needs pg_dump; the URL is in Railway → Postgres → Variables)
pg_dump "<DATABASE_PUBLIC_URL>" --no-owner -f simple_backup_YYYYMMDD.sql

# Raw-data safety net, per form:
#   Kobo → project → DATA → Downloads → XLS → Export
```

- Recommended cadence: a monthly DB dump plus XLSX exports of the Mother List, FSW Registration and the fistula registry, stored on a CIPRB drive.
- Three layers of truth exist: Kobo keeps every raw submission, every dashboard record stores its full raw payload, and the dumps above are the third copy.

7. Recurring duties calendar

When	Duty	Who
Daily / weekly	Work the approval queues (dashboard → Approvals). Unapproved records never reach the indicators.	Partner managers; CIPRB for surveillance forms
Monthly, first week	Generate and distribute the monthly report kit from the Reports page.	CIPRB M&E
Monthly	DB dump + key XLSX exports (section 6).	CIPRB IT / M&E

Monthly, the 13th	Glance at Railway usage against the $5 credit.	Railway account holder
Quarterly	Fistula campaign archive: append the new quarter (developer task, `fistula/campaign_archive.py`).	Developer + CIPRB RCH
Yearly	Domain renewal, auto-billed via Railway; confirm it went through.	Railway account holder

8. Commands a future developer will need

```
# Django management commands; run with the production env vars set.
python manage.py build_ciprb_forms  --only <form_id> --upload  # rebuild ONE CIPRB form
python manage.py build_bandhu_forms --upload                    # rebuild the 3 Bandhu forms
python manage.py build_phd_forms    --upload                    # rebuild the PHD forms
python manage.py export_fistula_clients --upload   # refresh the fistula patient lookup CSV
python manage.py resync_client_csvs                # resync every lookup CSV at once
python manage.py backfill_system_refs --apply      # stamp system refs on old records
```

> Four rules are baked into the codebase and must outlive every developer: raw payloads are never rewritten; IDs are issued by the server, never typed; every record lands PENDING until a human approves it; the dashboard and the reports read the same indicator service, so they can never disagree.